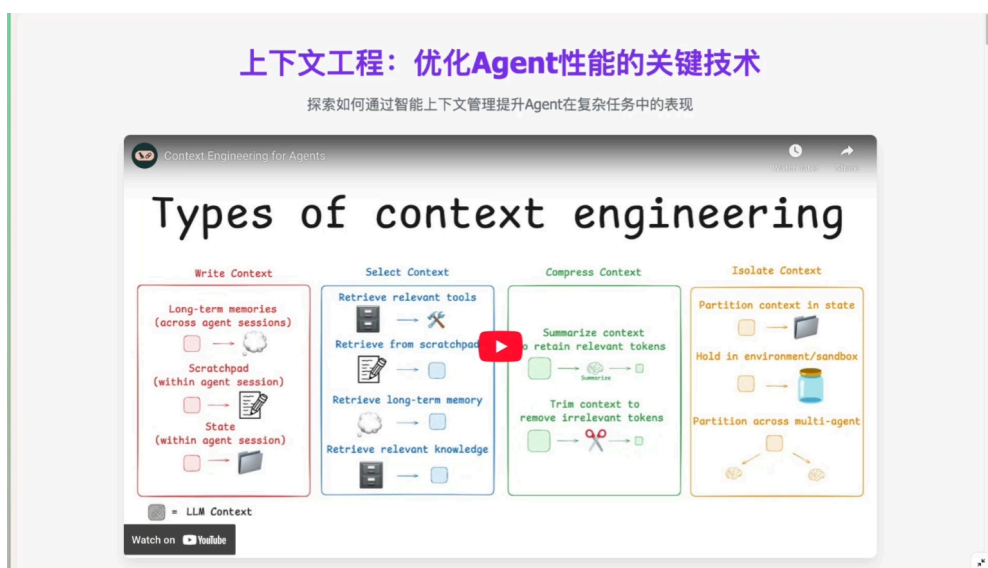


课程笔记

上下文工程：优化 Agent 效能的关键技术

沧海九粟 / 课程笔记

2026 年 4 月 10 日



视频作者/频道：沧海九粟

发布日期：2025-07-10

视频时长：22:28

视频链接：<https://www.bilibili.com/video/BV1w3GNzeEHb/>

目录

1 引言：从 Prompt Engineering 到 Context Engineering	3
1.1 一个争论引发的产业思考	3
1.2 LLM = CPU, Context Window = RAM	3
1.3 Prompt Engineering 为何已经不够用了	4
2 Context Engineering 的定义与四大策略框架	5
2.1 Context Engineering 的精确边界	5
2.2 LangChain 团队的四大策略框架	5
2.2.1 Write (存储上下文)	6
2.2.2 Select (选择上下文)	7
2.2.3 Compress (压缩上下文)	7
2.2.4 Isolate (隔离上下文)	7
2.3 从框架到语言：MCP 的产业信号	7
2.4 本章小结	8
3 存储上下文：把 RAM 放不下的内容落盘	8
3.1 Motivation：为什么需要把信息移出上下文窗口	8
3.2 Idea：两种存储形态——Scratchpad 与 Memory	8
3.2.1 Scratchpad：热存储与会话内草稿	8
3.2.2 Memory：持久化与跨会话记忆	9
3.3 Mechanism：Memory 的三类分类	9
3.3.1 Semantic Memory：事实性知识	9
3.3.2 Episodic Memory：经验与事件	10
3.3.3 Procedural Memory：偏好与操作指令	10
3.4 Takeaway：存储与 ReAct 的关联及工程启示	11
3.5 本章小结	11
4 选择上下文：把最相关的内容拉回 RAM	12
4.1 动机：提取比存储更重要	12
4.2 核心思想：按需召回，精准投喂	12
4.3 机制一：Memory 分类服务于选择性提取	12
4.4 机制二：Agentic RAG 与工具语义检索	13
4.5 选择策略的边界与注意	14
4.6 本章小结	14
5 压缩上下文：为 RAM 腾出空间	14
5.1 动机：RAM 总量有限，必须学会断舍离	14
5.2 核心思想：Summarize vs. Trim	14
5.3 机制一：Summarize——用 LLM 自身完成语义压缩	15
5.4 机制二：Trim——比 Summarize 更直接的空间释放	16
5.5 主动压缩 vs. 被动压缩：谁该掌握主动权？	17
5.6 从压缩到隔离：单窗口极限后的出路	17

5.7	本章小结	17
6	隔离上下文：用分治思想扩展有效 RAM	18
6.1	动机：单窗口的硬极限	18
6.2	核心思想：拆分与分发	18
6.3	状态隔离：子图与子 Agent	19
6.4	Sandbox 隔离：独立的执行环境	19
6.5	两种隔离方式的对比	20
6.6	本章小结	20
7	LangGraph 对上下文工程的原生支持	21
7.1	Motivation：从抽象策略到工程落地	21
7.2	Idea：状态图是上下文流转的天然载体	21
7.3	Mechanism：LangGraph 如何支撑四大策略	21
7.3.1	存储：store 与 state 的分层存储	22
7.3.2	选择：全局 state 与私有 state 的精准投喂	22
7.3.3	压缩：trim_messages、summarize 与自定义清理	23
7.3.4	隔离：subgraph、Tool Node 与 sandbox	23
7.4	LangSmith Trace：调试上下文流转的可视化抓手	23
7.5	Takeaway：把上下文策略写成可运行的状态图	24
7.6	本章小结	24
8	结论：Context Engineering 的系统思维与下一步实践	24
8.1	Motivation：Agent 效能的真正瓶颈在哪里	24
8.2	Idea：四大策略的协同闭环	25
8.3	Mechanism：从局部优化到系统思维	25
8.4	Takeaway：下一步实践建议	26

1 引言：从 Prompt Engineering 到 Context Engineering

1.1 一个争论引发的产业思考

在近期的 Agent 开发社区中，一个源自 Gavin 与 Anthropic 之间的技术争论引起了广泛讨论：我们是否真的需要多智能体系统？Gavin 持反对意见，认为构建多智能体并无必要，否则只会导致 *garbage in, garbage out*；而 Anthropic 则主张多智能体系统的必要性，并认为复杂的 Agent 协作是未来方向。无论是哪一方，当争论推进到具体实践时，双方的关注点最终都收敛到了同一个核心问题——**上下文的管理**。Andrew Karpathy (AK 老师) 顺势将这一讨论框架化，提出了 **Context Engineering** (上下文工程) 这一新兴概念，迅速成为 Agent 开发领域的主流关键词。

核心概念

Agent 效能瓶颈的根源往往不是模型能力本身，而是**上下文管理失当**。正如计算机科学中的古老谚语所言：*Garbage In, Garbage Out*——无论底层模型多么强大，如果塞入它上下文窗口的信息质量低下或不相关，其输出必然大打折扣。

这场争论也催生了一个重要的产业共识：与其继续争论架构形态，不如先建立一套系统化的上下文管理方法论。这也是本课程选择深入介绍 Context Engineering 的原因。接下来我们将基于 LangChain 团队的一篇高质量文章，结合 LangGraph 生态的实践工具，系统梳理上下文工程的定义、策略与落地方法。

1.2 LLM = CPU, Context Window = RAM

Andrew Karpathy 在阐述上下文工程时，借用了经典的操作系统隐喻，即 **LLM as OS**。他将大语言模型比作新型操作系统中的 **CPU**：它负责执行核心计算，而运算时所需的数据必须被加载到与之紧密耦合的内存中。这个内存就是 **Context Window** (上下文窗口)，相当于 CPU 旁边的 **RAM**。

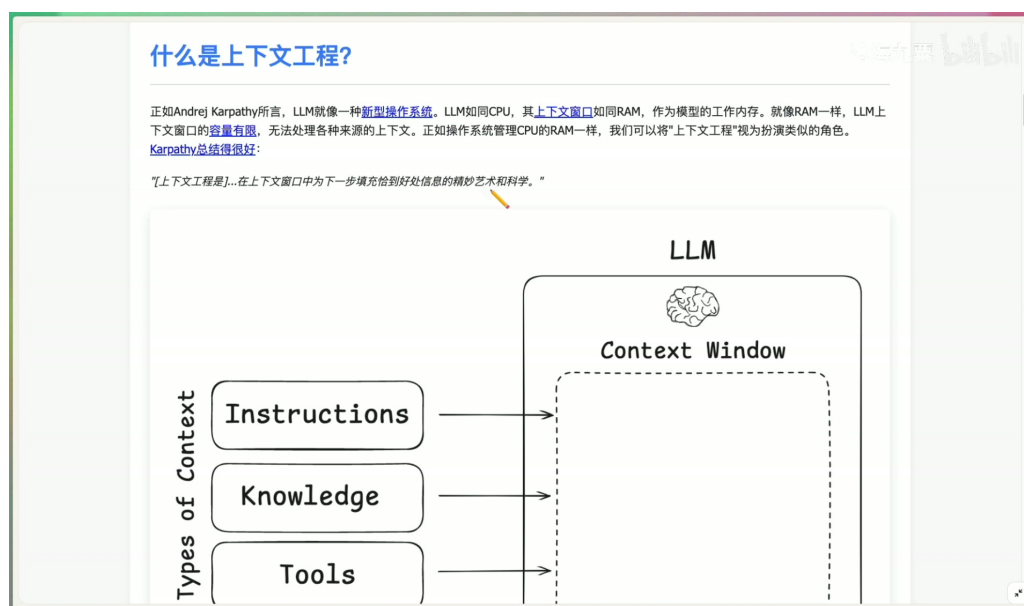


图 1: LLM 的 CPU-RAM 隐喻：Context Engineering 即管理上下文窗口这门“精妙的艺术和科学”¹

RAM 有两个关键特性与上下文窗口高度对应：第一，它与 CPU (LLM) 紧密结合，速度极快，模型可以直接在其上进行推理；第二，也是更关键的一点，**它的容量是有限的**。相比之下，磁盘或云存储可以近乎无限扩展，但 RAM 的大小始终存在硬边界。正因为这个硬边界的存在，我们才需要像操作系统管理内存一样，对上下文窗口进行系统的工程化管理。

核心概念

Context Engineering 可被定义为：在上下文窗口中为下一步操作填充恰到好处内容的精妙艺术和科学。这一定义将上下文管理从单纯的技巧提升到了系统工程的层面。

1.3 Prompt Engineering 为何已经不够用了

在 LLM 应用早期，开发者主要关注的是 **Prompt Engineering (提示工程)** ——通过精心编写 System Prompt、User Prompt 来提高模型输出质量。然而，随着 Agent 架构的演进，输入给模型的内容已经不再仅限于文字指令。

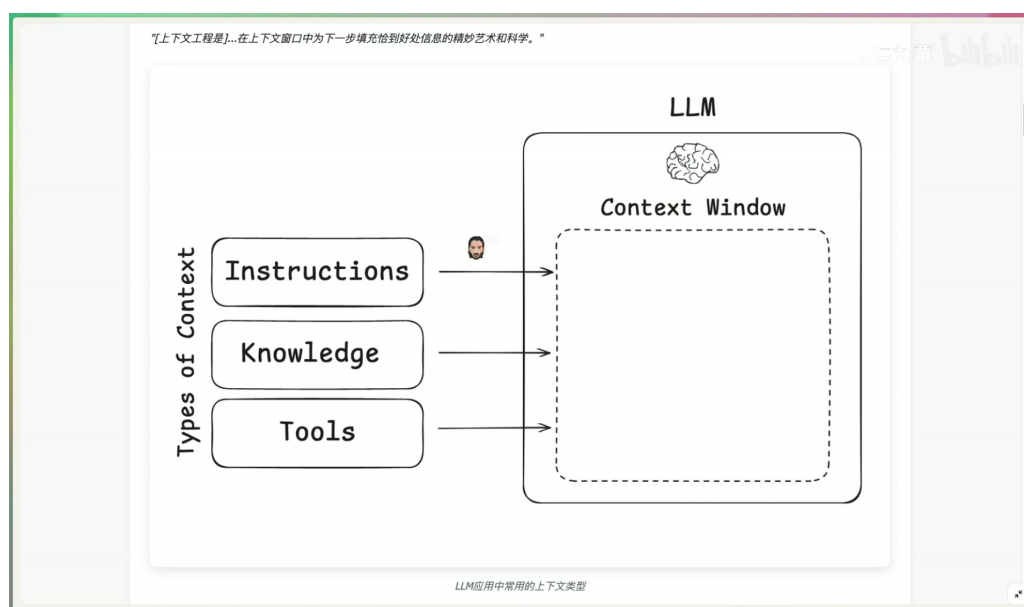


图 2: 上下文窗口中常见的三类内容：Instructions、Knowledge 与 Tools²

如上图所示，现代 Agent 的 Context Window 通常包含三大类信息：

1. **Instructions (指令)**：包括 System Prompt、User Prompt、Developer Message 等，直接指导模型行为。Prompt Engineering 主要作用于这一类。
2. **Knowledge (知识)**：通过 RAG (检索增强生成)、外部数据库或记忆系统注入的事实性信息。
3. **Tool I/O (工具输入/输出)**：Agent 调用工具时的参数以及工具返回的长文本结果，往往占据了上下文窗口的绝大部分空间。

¹ 视频画面时间区间：00:02:37-00:02:45。若为裁剪图，时间区间仍对应原视频画面。

² 视频画面时间区间：00:02:45-00:02:53。若为裁剪图，时间区间仍对应原视频画面。

常见误区

认为 Prompt Engineering 就是 Agent 调优的全部。事实上，提示工程只是 Context Engineering 的一个子集，它仅影响 **Instructions** 这一类内容。而现代 Agent 的效能更多地取决于如何管理 **Knowledge** 和 **Tool I/O** 的流入与流出。

正因如此，产业界的关键词正在从 Prompt Engineering 转向 Context Engineering。甚至更进一步，LangChain 团队提出了 **Agent Engineering** 的更大范畴；而近来火爆的 MCP (Model Context Protocol) 也反映出业界对“上下文”作为统一抽象层的高度关注。

2 Context Engineering 的定义与四大策略框架

2.1 Context Engineering 的精确边界

在厘清了 Prompt Engineering 与 Context Engineering 的关系之后，我们可以给出一个更精确的定义：

核心概念

Context Engineering (上下文工程) 是管理 **Instructions**、**Knowledge** 和 **Tool I/O** 三类信息的系统工程，其核心目标是在有限的上下文窗口中为模型下一步操作提供**恰到好处**的信息。

如果说 Prompt Engineering 只是在“调优指令”，那么 Context Engineering 则是在“调优一整台计算机的内存状态”。这不仅需要写好 prompt，还需要决定：哪些知识应该被写入窗口？哪些工具结果应该被保留？历史记录是否需要压缩？复杂任务是否应该拆分到多个子 Agent？凡此种种，都超出了传统提示工程的范畴。

2.2 LangChain 团队的四大策略框架

LangChain 团队基于上述 CPU-RAM 隐喻，将上下文工程系统化为四大策略。它们分别对应了操作系统管理内存的四种经典思路，形成一个完整的上下文管理闭环：

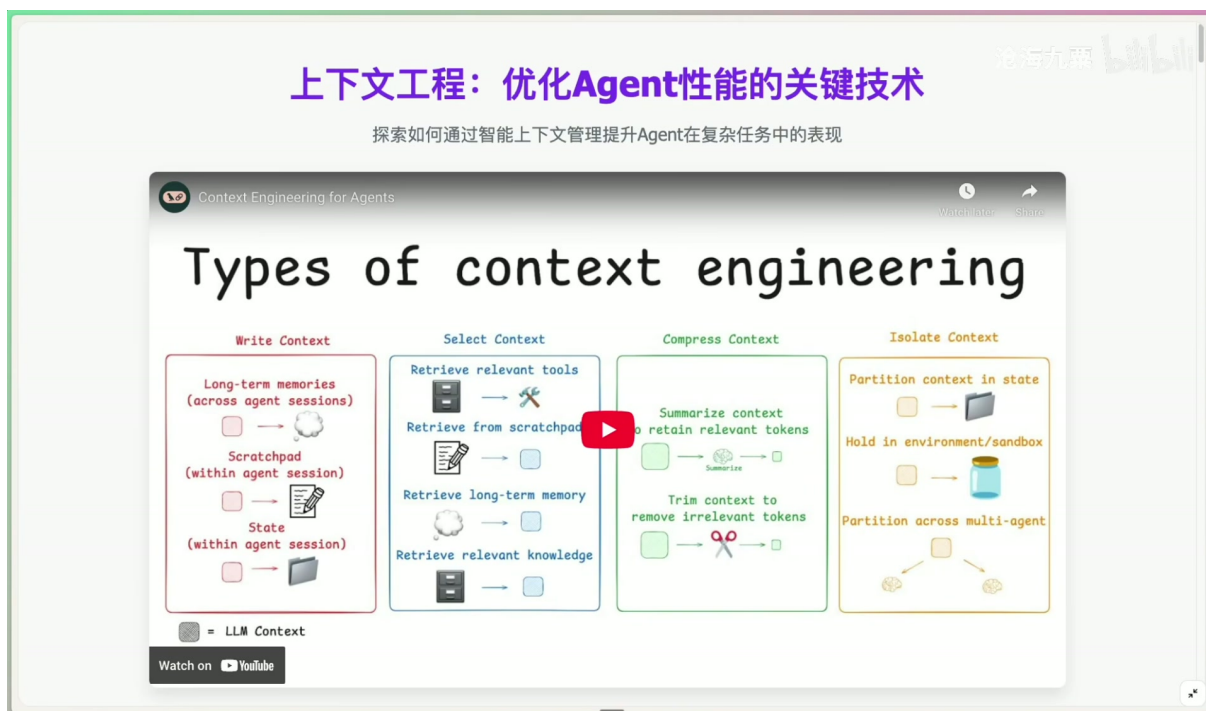


图 3: 上下文工程的四大策略：Write (Store)、Select、Compress 与 Isolate³

背景知识

将四大策略与操作系统/计算机架构中的概念进行类比，有助于建立直觉：

- **Write / Store Context** → **落盘**：当 RAM 放不下时，将数据写入磁盘或外部存储。
- **Select Context** → **加载**：根据当前任务需要，从磁盘将相关数据加载回 RAM。
- **Compress Context** → **清理**：通过摘要或裁剪释放 RAM 中的空间。
- **Isolate Context** → **分布式扩展**：一台机器的 RAM 不够用，就将任务分发到多台机器各自的 RAM 中处理，最后汇总。

下面我们对四大策略进行框架性的介绍，具体的实现细节将在后续章节逐一展开。

2.2.1 Write (存储上下文)

Write Context (更准确的说法是 **Store Context**) 的本质是：将暂时用不上但又有价值的信息保存到上下文窗口之外，以备后续使用。这对应操作系统中的“落盘”操作。常见的存储形态包括：

- **Scratchpad (临时笔记本)**：会话内的热存储，常用于记录中间推理步骤。
- **Memory (持久记忆)**：跨会话、甚至跨模型的长期存储，分为 Semantic、Episodic、Procedural 三类。

³视频画面时间区间：00:04:20-00:04:28。若为裁剪图，时间区间仍对应原视频画面。

2.2.2 Select (选择上下文)

Select Context 的本质是：在需要时，从外部存储或工具中将最相关的内容**拉回**上下文窗口。这对应操作系统中的”加载”操作。其关键挑战在于”相关性判断”——不仅要决定从哪里取 (scratch-pad / memory / tools)，还要决定取什么对当前任务最有价值。

Agentic RAG 本质上就是一种选择上下文的实践：Agent 主动搜索外部知识，将召回结果写入 Context Window，再供 LLM 推理。当工具数量庞大时，甚至需要先对工具进行语义检索，再决定调用哪些，以避免无关工具描述挤占窗口。

2.2.3 Compress (压缩上下文)

Compress Context 是在上下文窗口内容过多时，通过**摘要 (Summarize)** 或**裁剪 (Trim)** 来释放空间。这对应操作系统中的”内存清理”。两种核心手段的区别在于：

- **Summarize**：调用 LLM 对长文本进行语义压缩，保留关键信息，计算成本较高但信息损失较小。
- **Trim**：直接删除部分历史消息（如仅保留最近 N 条）或设定 TTL (Time-To-Live)，更为简单粗暴但效率更高。

Claude Code 中提供的 ‘/compact’ 主动压缩命令，以及 95% 阈值触发的被动压缩机制，都是业界典型的压缩实践。

2.2.4 Isolate (隔离上下文)

Isolate Context 是四大策略中解决根本瓶颈的终极手段。单台机器的 RAM (上下文窗口) 容量是绝对有限的，当存储、选择和压缩都不足以应对任务复杂度时，就需要将上下文**拆分**并分发给不同的子图、子 Agent 或 Sandbox 处理——这正是分布式计算中的”横向扩展”思想。其核心思想可概括为 **Divide-and-Conquer (分治法)**：拆分 → 并行/串行处理 → 结果汇总。

常见误区

认为压缩可以解决所有上下文长度问题。事实上，Compress 只能治标；当任务复杂度超过单窗口承载极限时，**Isolate** 才是根本的扩展之道。

2.3 从框架到语言：MCP 的产业信号

四大策略的提出不仅是学术框架，也得到了产业界的强烈呼应。MCP (Model Context Protocol) 的兴起就是一个明确的信号：产业界对上下文的关注正在从”提示词调优”转向”全栈上下文协议”。MCP 中”Context”的概念与本课程讨论的 Context 同属一个范畴——它试图统一模型与外部世界之间的上下文交换标准。

```
1 # 仅作概念示意，非可直接运行的完整代码
2 # Write / Store    -> store, state
3 # Select          -> node-level retrieval, RAG
4 # Compress        -> trim_messages, summarize
5 # Isolate         -> subgraph, sandbox tool node
```

Listing 1: 四大策略与 LangGraph 核心概念的对应关系（概念示意）

这也为后续第 7 节埋下了伏笔：LangGraph 的 ‘StateGraph’ 设计天然适配这四种策略——‘store’ 支撑存储，‘state’ 支撑选择，内置工具支撑压缩，‘subgraph’ 与 ‘Tool Node’ 支撑隔离。

2.4 本章小结

本章从 Gavin 与 Anthropic 的产业争论出发，引出了 **Context Engineering** 这一比 Prompt Engineering 更宏大的系统视角。我们建立了 **LLM = CPU, Context Window = RAM** 的核心隐喻，厘清了上下文窗口中 **Instructions**、**Knowledge**、**Tool I/O** 三类内容，并在此基础上系统介绍了 LangChain 团队提出的四大策略框架：

- **Write / Store**：将信息保存到窗口之外（落盘）。
- **Select**：在需要时将最相关内容拉回窗口（加载）。
- **Compress**：清理窗口空间，摘要或裁剪（Shrink）。
- **Isolate**：通过分治拆分到多个子单元处理（分布式扩展）。

接下来的章节将依次深入这四大策略的具体原理、方法与 LangGraph 中的落地实践。

3 存储上下文：把 RAM 放不下的内容落盘

3.1 Motivation：为什么需要把信息移出上下文窗口

在上一节建立的 LLM-as-OS 隐喻中，Context Window 被比作 CPU 旁的 RAM。RAM 的关键特性是“快”且“容量有限”。当 Agent 执行任务时，工具调用的输入输出、检索到的外部知识、多轮对话历史都会迅速填满这一有限空间。如果不把暂时用不上但仍有价值的信息及时移出 RAM，新内容将无法加载，Agent 的下一步推理也会因上下文截断而失真。因此，**Write/Store Context**（写入/存储上下文）的根本动因，就是在上下文窗口之外建立外部存储层，实现信息的“落盘”与后续按需召回。

核心概念

存储上下文 (Write/Store Context) 是指将暂时不需要直接参与当前推理、但又可能对后续步骤有价值的信息，保存在上下文窗口之外（如本地文件、内存对象、向量数据库或专用记忆存储），以便在需要时重新加载回上下文窗口供 Agent 使用。

3.2 Idea：两种存储形态——Scratchpad 与 Memory

视频中将外部存储划分为两种典型形态：一种是**临时笔记本 (Scratchpad)**，另一种是**持久记忆 (Memory)**。理解二者的区别，是设计 Agent 存储策略的第一步。

3.2.1 Scratchpad：热存储与会话内草稿

Scratchpad 的概念最早来自 ReAct (Reasoning + Acting) 范式。在早期的 ReAct 实现中，Agent 会在文本补全的循环中维护一个 ‘scratchpad’ 字段，用于记录中间推理步骤、观察结果和行动计划

ootnote 图截自视频 06:08。。当时的 ‘scratchpad’ 是存在于上下文窗口**内部**的；而今天的 Context Engineering 更强调把它**放到窗口之外**——例如写入文件、工具调用结果字段，或持久化的运行时状态对象中。

背景知识

ReAct 是一种将推理 (Reasoning) 与行动 (Acting) 交替进行的 Agent 范式。在其原始论文与实现中，Agent 会把每一步的“思考 (Thought) → 行动 (Action) → 观察 (Observation)”写进一个称为 **scratchpad** 的文本区域，作为下一轮推理的上下文基础。这可以被视为现代上下文管理理念的前身。

Scratchpad 的本质特征有三点：

- **临时性**：通常绑定于单次会话或单条任务链路，任务结束后即可清理；
- **热存储**：距离 LLM 很近，延迟低，能够快速读写；
- **局部性**：常与单个 Agent 实例绑定，不强调跨会话共享。

以 Claude Code 为例，它在工作目录下维护的 ‘claude.md’ 文件就扮演了类似 Scratchpad 的角色：Agent 可以把当前会话中总结出的一些关键信息写入该文件，在后续需要时先加载它，再展开新的推理。

3.2.2 Memory：持久化与跨会话记忆

与 Scratchpad 相对，**Memory** 的核心判据是**跨会话 (cross-session)**，甚至可以跨模型、跨 Agent

ootnote 图截自视频 07:05。。Memory 强调信息不因单次对话结束而丢失，而是在不同的用户会话、不同的 Agent 实例之间保持可用，从而支持个性化输出和长期行为一致性。

核心概念

判断一段存储是 Scratchpad 还是 Memory 的关键标准在于：

- **Scratchpad**：会话内、临时、热存储；
- **Memory**：跨会话、持久、冷存储，服务于长期个性化与经验累积。

3.3 Mechanism：Memory 的三类分类

为了让“落盘”的信息在后续“选择 (Select)”阶段能够被高效召回，LangChain 生态将 Memory 进一步细分为三大类：**Semantic Memory (语义记忆/事实记忆)**、**Episodic Memory (情景记忆/经历记忆)** 与 **Procedural Memory (程序性记忆/指令记忆)**

ootnote 图截自视频 08:20。。这种分类并非纯学术研究，而是直接服务于工程实践中的检索与召回策略。

3.3.1 Semantic Memory：事实性知识

Semantic Memory 用于存储与任务或用户相关的**事实**。在人类认知中，它对应“我在学校里学到的东西”；在 Agent 系统中，它对应“关于用户的事实”或“从外部知识库中沉淀下来的结构化知识”。例如：

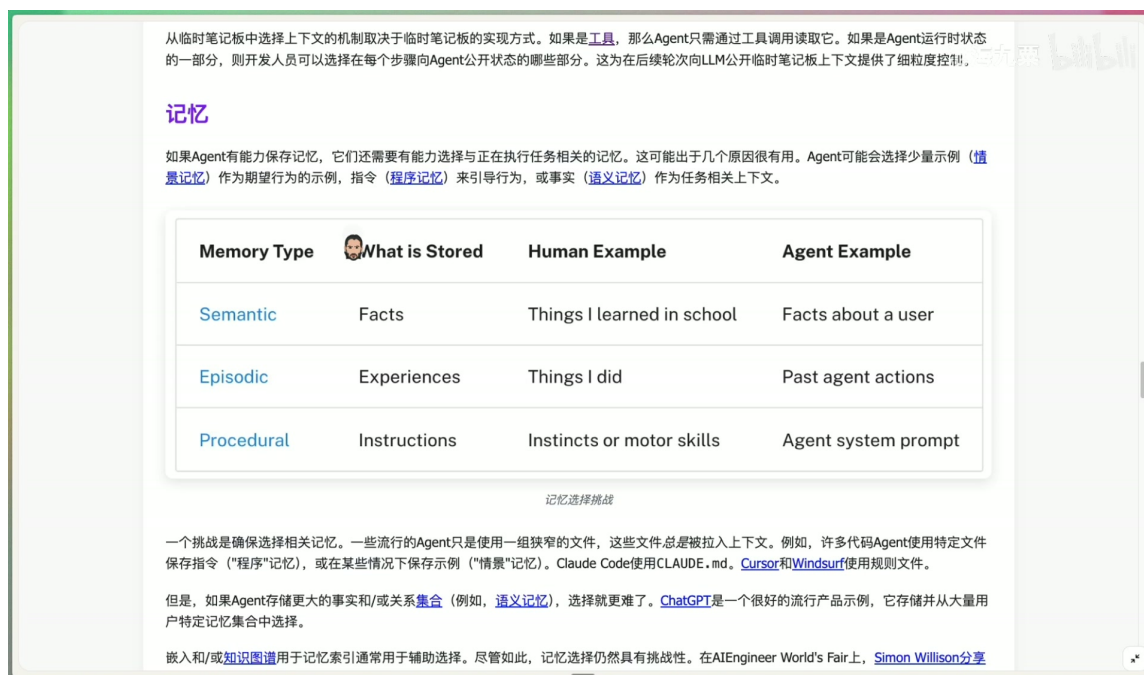


图 4: Memory 的三类分类: Semantic、Episodic、Procedural 的定义与示例对比。

- 用户的职业、所在城市、技术栈偏好；
- 经过 RAG 检索并验证后的产品文档摘要；
- 长期积累的领域概念图谱。

3.3.2 Episodic Memory: 经验与事件

Episodic Memory 记录 Agent 曾经历过的事件与交互历史。在人类认知中，它对应“我做过的事情”；在 Agent 系统中，它对应“过去的 Agent 行为记录”。例如：

- 某次成功调试 bug 的完整会话轨迹；
- 用户在上一轮融资中提出的具体问题与反馈；
- Agent 在不同任务中的失败案例及其根因分析。

这类记忆对于少样本学习（few-shot prompting）和反思（reflection）尤为重要：当 Agent 遇到类似问题时，可以从 Episodic Memory 中调取历史示例作为参考。

3.3.3 Procedural Memory: 偏好与操作指令

Procedural Memory 存储用户的偏好、习惯或操作指令。在人类认知中，它对应“本能或运动技能”；在 Agent 系统中，它对应“Agent system prompt”或用户反复要求的输出格式。例如：

- 用户要求所有返回的代码必须附带单元测试；
- 用户偏好的报告模板、术语集或注释风格；
- 某个项目特定的编译命令或部署流程。

```

1 # 概念示意，非完整可运行代码
2 memory_store = {
3     "semantic": {
4         "user_location": "Shanghai",
5         "tech_stack": ["Python", "LangGraph"]
6     },
7     "episodic": [
8         {"event": "debugged_race_condition", "outcome": "success", "date": "
9             2025-03-10"}
10    ],
11    "procedural": {
12        "output_format": "Always include a unit test.",
13        "comment_style": "Google Docstring"
14    }
15 }

```

Listing 2: Memory 分类在伪代码中的示意

3.4 Takeaway: 存储与 ReAct 的关联及工程启示

从 ReAct 的 ‘scratchpad’ 到现代 Agent 的三层 Memory (Semantic / Episodic / Procedural)，上下文存储的演进揭示了一条清晰脉络：上下文管理正从“临时草稿”走向“结构化持久化”。这一演进带来的关键启示包括：

1. **先存后取**：存储上下文的直接目的是为后续“选择上下文 (Select Context)”提供原料；没有良好的分类存储，就没有精准的召回。
2. **分类服务于召回**：Memory 的三类划分不是为了学术概念，而是为了在特定情境下提高提取的有效性。例如，当用户询问“我之前怎么做的”时，系统应优先搜索 Episodic Memory；当用户要求“按我习惯的方式输出”时，应优先读取 Procedural Memory。
3. **Scratchpad 仍未过时**：对于单次会话内的中间结果、思考链、临时计划，Scratchpad 依然是最轻量的热存储方案，Claude Code 的 ‘claude.md’ 机制就是现实例证。

3.5 本章小结

本节围绕“存储上下文”这一策略展开，核心内容可总结为以下三点：

- **两种存储形态**：Scratchpad 是会话内的临时热存储，源自 ReAct 范式；Memory 是跨会话的持久冷存储，强调长期可用与跨模型共享。
- **Memory 的三类分类**：Semantic Memory 存储事实，Episodic Memory 存储经验与事件，Procedural Memory 存储用户偏好与操作指令。分类的目的在于提升后续选择 (Select) 阶段的召回精度。
- **与 ReAct 的关联**：现代上下文存储机制是对 ReAct ‘scratchpad’ 的工程化扩展——从把草稿写在窗口里，到把草稿写在文件里，再到把记忆结构化地落盘并分类管理。

在下一节（选择上下文）中，我们将讨论如何把本节“落盘”的内容，在恰当的时机以最高的相关性重新加载回上下文窗口。

4 选择上下文：把最相关的内容拉回 RAM

4.1 动机：提取比存储更重要

在上一节中，我们讨论了如何把放不下的上下文先“落盘”到外部存储。然而，存储只是第一步——如果 Agent 无法在恰当的时刻把最相关的内容捞出来，那么再完美的记忆分层也只是沉睡的数据。正如视频中强调的，“你不能光存，取出来的东西也必须对当前问题有效”。

现代 Agent 的上下文来源早已不限于单一会话历史。除了临时笔记本（Scratchpad）和跨会话的持久记忆（Memory），工具调用（Tool Use）也日益成为上下文的重要入口。如何从这三类来源中精准地拉回与当前任务最匹配的信息，正是“选择上下文”（Select Context）这一策略要解决的核心问题。

4.2 核心思想：按需召回，精准投喂

选择上下文的本质可以概括为一句话：**在需要的时刻，将与当前情境最相关的内容加载进上下文窗口**。这听起来像是操作系统将磁盘页换入 RAM 的过程，但它比简单的分页调度更复杂——因为“相关性”本身是语义层面的，需要借助检索或推理能力来判断。

核心概念

Select Context 不仅决定“从哪里取”（Scratchpad / Memory / Tools），还决定“取什么对当前任务最有价值”。它的目标是让 LLM 在每一步推理时，上下文窗口里都恰好装满“恰到好处的内容”。

4.3 机制一：Memory 分类服务于选择性提取

在 LangChain 生态中，Agent 的记忆通常被划分为三类：

- **Semantic Memory（语义记忆）**：存储事实性知识，如用户的基本信息、领域常识。
- **Episodic Memory（情景记忆）**：记录发生过的具体事件与交互经历，如过去的会话片段。
- **Procedural Memory（程序性记忆）**：存储用户的偏好、习惯与操作流程，如常要求的输出格式。

这三类分类并非为了学术研究，而是为了服务于**提取精度**。当 Agent 收到一个新查询时，它可以根据查询类型首先判断应该优先检索哪一类记忆：

- 询问“我是谁”或项目背景 → 优先查 Semantic Memory。
- 询问“我们之前怎么做的” → 优先查 Episodic Memory。
- 询问“按我喜欢的格式输出” → 优先查 Procedural Memory。

图 5 展示了视频中出现的记忆类型对照表。通过将记忆按类型分离存储，Agent 在选择阶段就能缩小搜索范围，降低无关信息挤占上下文窗口的风险。

⁴视频画面时间区间：00:08:15–00:08:30。若为裁剪图，时间区间仍对应原视频画面。

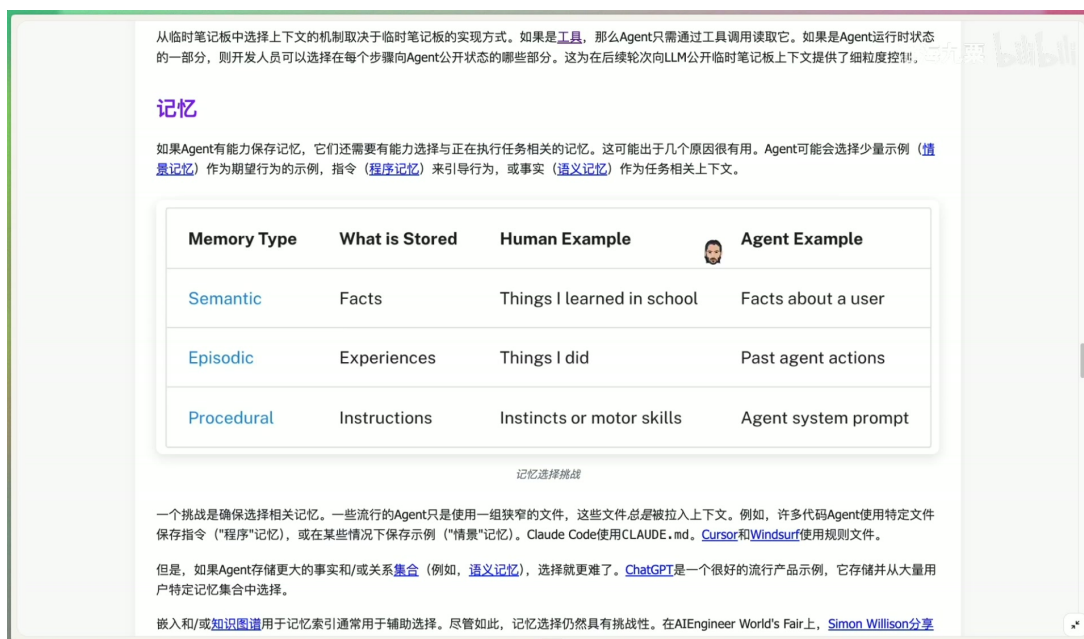


图 5: Memory 的三类分类: Semantic、Episodic、Procedural 分别对应事实、经历与指令。⁴

4.4 机制二: Agentic RAG 与工具语义检索

除了从记忆中提取, Agent 还越来越多地通过工具调用来获取外部知识。这就引出了视频中提到的另一个关键概念——Agentic RAG。

核心概念

Agentic RAG 本质上是“通过工具搜索外部知识, 并把召回结果写入上下文窗口”。与传统 RAG 不同, Agent 不是被动接收检索结果, 而是主动决定何时调用搜索工具、如何过滤结果、再将其注入上下文。

然而, 当 Agent 可用的工具数量变得非常庞大时, 一个新的问题出现了: 工具描述本身就可能塞满上下文窗口。为此, LangChain 团队提供了 big tools (preview) 类库, 专门用于在调用前对工具进行语义检索——只召回与当前查询最相关的那一部分工具, 避免无关工具的 schema 和说明文字白白占用宝贵的 token。

以下伪代码示意了这一流程:

```
1 # 步骤 1: 根据查询语义检索相关工具
2 selected_tools = tool_search(query, available_tools, top_k=5)
3
4 # 步骤 2: 使用筛选后的工具进行 RAG 检索
5 retrieved_docs = rag_retrieve(selected_tools, query)
6
7 # 步骤 3: 将召回内容写入上下文窗口, 供 LLM 推理
8 context_window = build_context(instructions, retrieved_docs, chat_history)
9 response = llm.generate(context_window)
```

Listing 3: Agentic RAG 与工具语义检索概念示意

背景知识

在传统的单轮 RAG 中，检索系统通常一次性返回若干文档片段。而 Agentic RAG 把检索本身也变成了 Agent 的一个行动步骤：Agent 可以决定调用哪个搜索工具、以什么关键词搜索、甚至对检索结果进行二次筛选后再写入上下文。这种“主动召回”的能力，使上下文内容的质量显著高于被动注入。

4.5 选择策略的边界与注意

需要强调的是，“选择上下文”并非万能药。它的前提是：**被选择的内容总量仍然要小于上下文窗口的剩余容量**。如果我们精心挑选后召回的结果依然过长——例如一段巨大的代码 diff、一篇超长的技术文档——那么下一步就必须引入压缩策略。这也自然地引出了我们下一节的主题。

常见误区

误以为“只要做了 RAG 或 Memory，上下文管理就万事大吉”。实际上，如果召回的文档块过大、或工具返回结果未经裁剪，选择回来的内容仍然可能瞬间撑满窗口，导致模型推理质量下降。选择（Select）与压缩（Compress）往往是配套使用的。

4.6 本章小结

本节从“提取比存储更重要”的动机出发，介绍了选择上下文（Select Context）策略的两条核心机制：

1. **Memory 分类支持精准提取**：通过 Semantic / Episodic / Procedural 的层次化存储，Agent 可以根据查询类型匹配到正确的记忆层级，提高召回效率。
2. **Agentic RAG 与工具语义检索**：Agent 主动调用工具搜索外部知识，并通过“big tools”等语义检索机制避免无关工具描述挤占窗口。

然而，即使召回内容经过精心选择，仍可能超出单窗口的容量极限。因此，在将内容注入 LLM 之前，我们往往还需要对上下文进行**压缩**——这正是下一节要讨论的核心手段。

5 压缩上下文：为 RAM 腾出空间

5.1 动机：RAM 总量有限，必须学会断舍离

上一节我们讨论了如何把最相关的内容从外部存储“换入”上下文窗口。但在实际开发中，即便经过精心挑选，换入的内容仍然可能超出窗口容量——尤其是当 Agent 执行长会话、调用返回大量 token 的工具、或需要同时参考多篇文档时。

正如引言中建立的 LLM-as-OS 隐喻所示，**上下文窗口的容量始终存在硬边界**。当换入内容触及这一上限时，就必须有意识地清理窗口，为后续推理腾出空间。这就是“压缩上下文”（Compress Context）策略存在的意义。

5.2 核心思想：Summarize vs. Trim

压缩上下文的手段可以归纳为两大类：

- **Summarize (摘要)**: 调用 LLM 对长内容进行语义压缩，保留关键信息，减少 token 占用。
- **Trim (裁剪)**: 通过硬规则直接删除部分历史消息或过期内容，快速释放空间。

两者的核心差异在于：Summarize 追求“内容的语义保真”，而 Trim 追求“空间释放的速度与确定性”。选择哪种策略，取决于被压缩信息的“价值密度”——高价值信息优先 Summarize，低价值或已过时信息可直接 Trim。

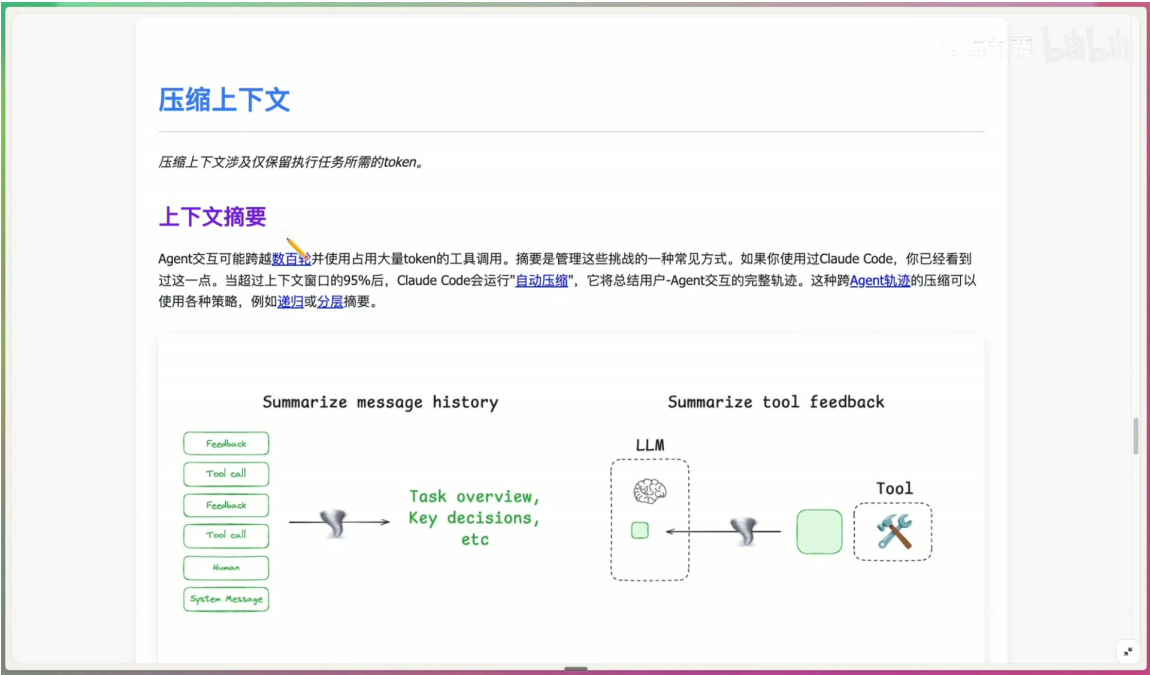


图 6: 上下文摘要的两个典型场景：压缩消息历史（左）与压缩工具返回结果（右）。⁵

5.3 机制一：Summarize——用 LLM 自身完成语义压缩

Summarize 的本质是让 LLM 把一篇长文本“读薄”。视频中特别指出，压缩的对象不限于对话历史，还包括：

- 多轮交互产生的超长消息链；
- 占用大量 token 的工具返回结果（如搜索、日志、代码 diff）；
- 需要跨 Agent 传递的中间状态。

图 6 清晰地展示了两个典型的摘要应用场景：左侧是将整条消息历史压缩为“Task overview, Key decisions, etc”；右侧是将 bulky 的工具反馈经过摘要后再传递给 LLM。

```
1 from langmem import summarize_history
2
3 # 当历史消息超过阈值时，触发摘要压缩
4 summary, remaining_messages = summarize_history(
5     messages=chat_history,
6     max_tokens=4000,
```

⁵ 视频画面时间区间：00:10:35–00:10:50。若为裁剪图，时间区间仍对应原视频画面。


```
7 instruction="保留任务目标、关键决策与未完成的行动项"
8 )
```

Listing 4: LangMem 对话历史摘要概念示意

在实际产品中，**Claude Code** 是 Summarize 策略的业界标杆。它提供了两种压缩触发模式：

- **主动压缩**：用户通过 `/compact` 命令手动触发，Claude Code 会对当前会话历史进行摘要并释放空间。
- **被动压缩**：当上下文窗口的使用率达到 **95%** 时，系统会自动触发压缩，无需用户干预。

核心概念

Claude Code 的 95% 被动压缩阈值是一个极具参考价值的设计：它既避免了在窗口尚未紧张时频繁浪费计算资源做摘要，又能在真正触及上限前留出安全缓冲，保证后续推理仍有可用的 token 余量。

5.4 机制二：Trim——比 Summarize 更直接的空间释放

当会话历史过于冗长、或部分内容已经明确失去时效性时，直接删除往往比调用 LLM 做摘要更经济、更快速。视频中提到的 Trim 手段包括：

- **保留最近 N 条消息**：一种最简单也最常见的硬裁剪策略。
- **Time-To-Live (TTL)**：为消息或记忆设定存活时间，过期后自动从上下文中剔除。
- **节点级状态裁剪**：在 LangGraph 等框架中，可以在特定节点显式删除不再需要的 state 字段。

图 7 展示了视频中关于“上下文修剪”的说明：除了用 LLM 提取摘要，还可以直接过滤掉较早的消息，实现轻量级的上下文瘦身。

```
1 from langchain_core.messages import trim_messages
2
3 # 仅保留最近 N 条消息，或按 token 上限裁剪
4 trimmed = trim_messages(
5     messages=chat_history,
6     strategy="last",
7     max_tokens=6000,
8     token_counter=len # 实际场景中应替换为 tokenizer
9 )
```

Listing 5: LangGraph 中 Trim 裁剪的消息过滤概念示意

常见误区

混淆 Summarize 与 Trim 的适用场景。Summarize 更适合高价值、需要保留语义的信息；Trim 更适合低价值、已过期、或仅需保留局部时序的信息。错误地在关键决策链上使用 Trim，可能导致模型丢失必要的推理依据。

⁶视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

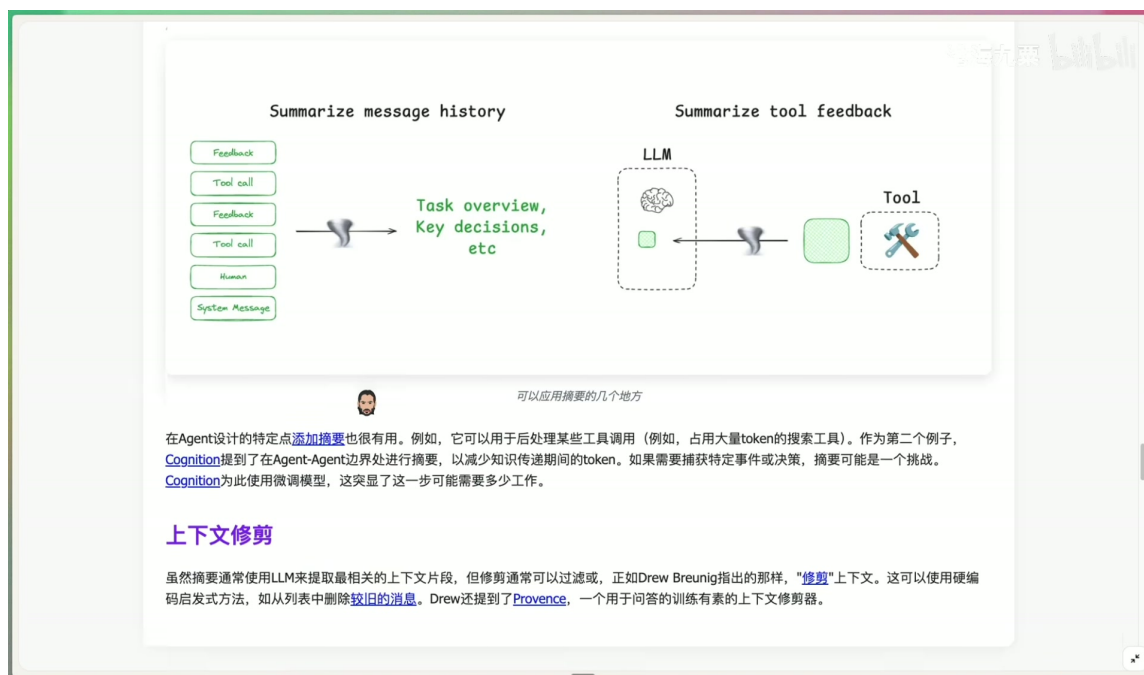


图 7: 上下文修剪 (Trim) 作为 Summarize 的补充策略，可通过删除较旧消息快速释放空间。⁶

5.5 主动压缩 vs. 被动压缩：谁该掌握主动权？

压缩的触发方式也值得系统设计时仔细斟酌。视频以 Claude Code 为例，给出了两条明确的路径：

1. **主动压缩**：用户明确知道窗口即将吃紧，通过 `/compact` 主动发起。优点是摘要目标清晰、用户可控；缺点是需要人时刻关注窗口使用情况。
2. **被动压缩**：系统监测到窗口使用率达到阈值（如 95%）后自动执行。优点是无感、鲁棒；缺点是自动摘要可能在某些边界场景丢失关键细节。

在构建自己的 Agent 时，一个务实的做法是**两者兼备**：设置被动压缩作为安全网，同时保留主动压缩的入口，让用户或上层调度器在关键节点介入。

5.6 从压缩到隔离：单窗口极限后的出路

压缩能够在一定程度上延缓上下文窗口的耗尽，但它终究是**单窗口内部的腾挪**，无法突破 RAM 的物理上限。当任务复杂到一定程度——例如需要同时处理多份超长文档、执行数百轮工具调用、或协调多个并行子任务时——即使最好的压缩算法也无能为力。

这正是下一章“隔离上下文”所要解决的问题：**既然单台机器的 RAM 有极限，何不把任务拆分给多台机器？**压缩是治标，隔离（分治）才是扩展有效处理能力的根本之道。

5.7 本章小结

本节围绕“为 RAM 腾出空间”的目标，系统阐述了压缩上下文 (Compress Context) 策略的核心机制：

1. **Summarize (摘要)**：通过 LLM 对消息历史、工具结果等进行语义压缩，保留核心价值；Claude Code 的 `/compact` 与 95% 自动阈值是业界典型实现。

2. **Trim (裁剪)**: 通过保留最近 N 条、TTL 过期、节点级状态删除等硬规则快速释放空间，是 Summarize 的有力补充。
3. **主动 vs. 被动压缩**: 建议两者结合，既保证用户体验的可控性，又设置自动安全网防止窗口溢出。

压缩与选择形成了一对紧密配合的上下文管理动作：**选择确保拉回来的内容足够相关，压缩确保这些内容仍然装得下 RAM**。然而，当任务本身的规模超越单窗口承载极限时，我们就必须从“内部腾挪”转向“外部扩展”——引入隔离策略，从“单窗口管理”跃迁到“多窗口分布式扩展”。

6 隔离上下文：用分治思想扩展有效 RAM

6.1 动机：单窗口的硬极限

前文中我们讨论了存储 (Write)、选择 (Select) 与压缩 (Compress) 三种策略。它们的核心假设是：所有信息最终仍需汇入同一个上下文窗口，我们只是在这个窗口内“腾挪空间”。然而，正如引言中建立的 LLM-as-OS 隐喻所示，上下文窗口的容量存在绝对上限——无论怎么压缩，RAM 的尺寸不变。

核心概念

当单上下文窗口的容量成为不可逾越的瓶颈时，**隔离 (Isolate)** 是唯一的根本解法：把任务拆分给多个执行单元，让每个单元只处理与其相关的局部上下文，从而横向扩展系统的有效处理能力。

这就好比单机内存不足时，最终方案不是无限清理内存，而是把任务分发到多台机器上并行处理。对于 Agent 系统而言，隔离的本质正是**分治法 (Divide-and-Conquer)**：拆分 → 并行/串行处理 → 结果汇总。

6.2 核心思想：拆分与分发

隔离上下文 (Isolate Context) 的做法是将原本聚合在一起的上下文拆分开，分发给不同的子 Agent、子图 (subgraph) 或执行环境。视频在讲解这一策略时指出，这种方式虽然名为“隔离”，实则更具**扩展性**：

- 不同的 Agent 可以运行在不同的进程、容器甚至服务器上；
- 每个 Agent 基于被优化过的、更适合自身任务的局部上下文进行操作；
- 全局的信息丰富度并未丢失，但每个局部的上下文更加优质、适配性更强。

OpenAI 的 Swarm 库以及 Anthropic 在多 Agent 系统上的研究，其核心动机都在于此——通过**关注点分离 (Separation of Concerns)**，让每个 Agent 持有特定的工具集、指令和专属的上下文窗口。

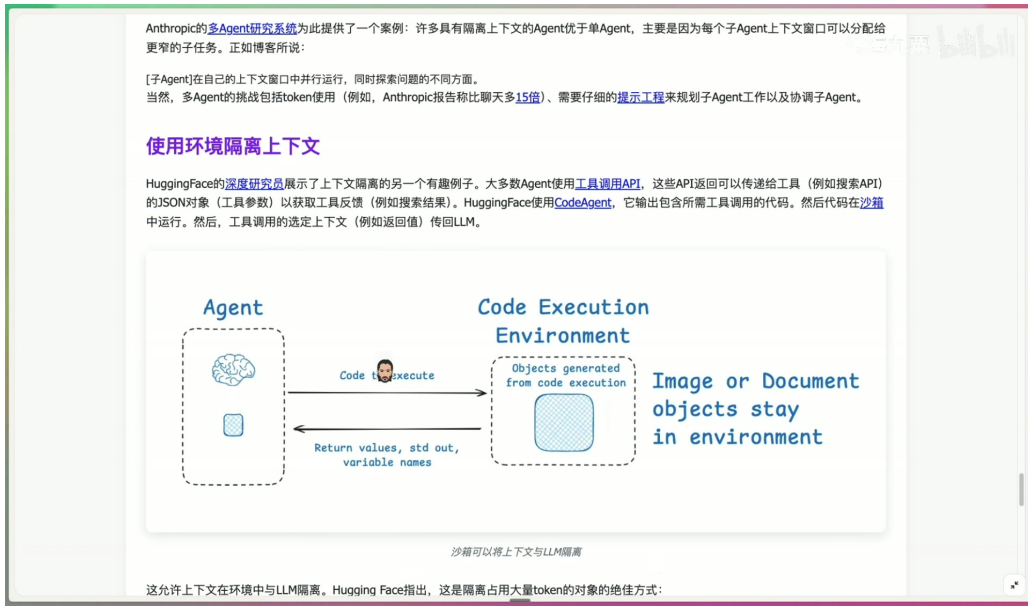


图 8: 隔离上下文与多 Agent 架构：将全局上下文拆分到不同的子 Agent 中，每个 Agent 保有独立的工具与状态。

6.3 状态隔离：子图与子 Agent

状态隔离是最常见的一种隔离方式，其做法是将全局状态拆分为多个局部状态，每个执行单元仅访问与自身任务相关的子集。在 LangGraph 中，这表现为**子图（subgraph）**机制：父图维护全局 state，而子图拥有独立的局部 state，子图运行完毕后可将结果回写至上层 state。

状态隔离不一定要写入磁盘。即使在内存中进行，只要不同 Agent 或节点之间的 state 访问范围被明确划分，就已经达成了隔离目标。视频指出，这种方式的隔离通常涉及**纯粹的数据传递**——例如格式化的查询条件、摘要后的中间结果、分片后的任务描述等。

```
1 # 概念示意（伪代码）
2 parent = StateGraph(GlobalState)
3 parent.add_node("subgraph_a", subgraph_a.compile())
4 parent.add_node("subgraph_b", subgraph_b.compile())
5
6 # subgraph_a 与 subgraph_b 各自拥有独立的局部 state
7 # 仅将所需子集从 GlobalState 注入，再回写结果
```

Listing 6: 状态隔离的概念示意：父图将子任务路由到不同的子图

子 Agent 的状态隔离思路与此完全一致。Swarm 等多 Agent 框架通过 **Agent 间切换（handoff）**实现上下文拆分：当一个子任务更适合由另一个专家 Agent 处理时，仅传递该 Agent 所需的上下文子集，主 Agent 不必背负全部历史消息。

6.4 Sandbox 隔离：独立的执行环境

与状态隔离相比，**Sandbox（沙盒）隔离**更进一步：它不仅拆分数据，还提供了一个完全独立的执行环境。典型场景是向 Sandbox 传入大量代码或外部资源，让其在隔离环境中运行，最终只将运行结果（如 stdout、返回值、错误摘要）返回给主 Agent。

背景知识

Sandbox 是一种受控执行环境，常用于运行不可信或资源密集型的代码。在 Agent 系统中，Sandbox 隔离的核心价值在于：把巨大的中间产物（如完整代码库、执行日志、临时文件）挡在主上下文窗口之外。

视频特别强调，Sandbox 隔离与状态隔离的最大区别在于上下文的内容形态：

- **状态隔离**：传递的多为结构化数据、任务参数、查询条件；
- **Sandbox 隔离**：传递的往往是大量代码，Sandbox 负责编译或执行，再把精简后的结果传回。

举例来说，当 Agent 需要分析一个大型代码仓库时，与其把成百上千个文件全部塞进上下文窗口，不如将这些文件交给代码分析 Sandbox，Sandbox 运行静态分析脚本后，仅返回关键函数调用图、依赖关系摘要等结论性内容。

```
1 # 概念示意（伪代码）
2 code_context = load_entire_repo() # 大量代码，远超窗口容量
3 result = sandbox.run(
4     script="analyze_deps.py",
5     context=code_context
6 )
7 # result 可能只有几百个 token， summaries 干净的分析结论
8 main_agent.invoke(result)
```

Listing 7: Sandbox 隔离的概念示意：主 Agent 只接收执行结果

6.5 两种隔离方式的对比

状态隔离与 Sandbox 隔离并不是互斥的，它们适用于不同的场景，也可以在同一个系统中组合使用。表 1 对比了两种隔离方式的关键差异。

表 1: 状态隔离与 Sandbox 隔离的对比

对比维度	状态隔离	Sandbox 隔离
隔离层级	数据/状态	数据 + 执行环境
典型载体	子图、子 Agent	代码沙盒、容器、虚拟机
传递内容	结构化数据、参数、摘要	大量代码、文件、待执行脚本
返回内容	局部状态更新、中间结果	执行结果、stdout、错误摘要
适用场景	多步骤任务拆分、Agent 协作	代码执行、资源密集型运算

无论采用哪种方式，隔离的本质都是一致的：把上下文分散到独立的可执行单元中处理，再汇总结果。这是一种典型的分治法模式，也是从“单窗口优化”迈向“分布式上下文工程”的关键一步。

6.6 本章小结

- **隔离**是突破单上下文窗口硬极限的根本策略，其核心思想是**分治**：拆分全局上下文，分发到多个子 Agent、子图或 Sandbox 中处理。

- **状态隔离**通过子图或子 Agent 实现，仅传递与当前任务相关的局部 state；OpenAI Swarm 和 Anthropic 多 Agent 研究是该思路的代表。
- **Sandbox 隔离**提供独立执行环境，常用于大量代码传递场景；主 Agent 只需接收运行结果，避免完整代码库常驻窗口。
- 两种隔离方式可以组合使用，共同将系统的有效 RAM 从“单个上下文窗口”扩展到“多个独立执行单元的局部窗口之和”，在保留全局信息丰富度的同时，显著提升每个局部的上下文质量与适配性。

7 LangGraph 对上下文工程的原生支持

7.1 Motivation：从抽象策略到工程落地

在前几节中，我们系统梳理了上下文工程的四大策略：**Write/Store**（存储）、**Select**（选择）、**Compress**（压缩）与 **Isolate**（隔离）。然而，对于开发者而言，一个更迫切的问题是：这些策略如何在实际代码中被系统性地落地？如果缺乏统一的技术载体，四大策略很容易沦为各自为政的补丁，甚至互相冲突。幸运的是，LangChain 生态中的 **LangGraph** 恰好提供了与这四大策略天然契合的图结构抽象。

核心概念

LangGraph 并不是在四大策略提出之后才“打补丁”式地增加功能；相反，它的 **StateGraph**、节点（Node）、子图（Subgraph）与存储（Store）设计，本身就将以“上下文流转与管理”视为一等公民。

7.2 Idea：状态图是上下文流转的天然载体

传统 Agent 框架往往将上下文隐式地堆叠在聊天记录或 prompt 字符串中，开发者难以精确控制哪些信息在何时进入模型的视野。LangGraph 的核心创新在于引入了 **StateGraph（状态图）**：上下文被显式地建模为一个结构化的 **state** 对象，在图的节点与边之间有序流转。

背景知识

LangGraph 的每个节点本质上是一个函数，它接收当前 **state** 并返回更新后的 **state**。由于 **state** 的透明性，开发者可以在任意节点中读取、筛选、压缩或拆分上下文，而无需依赖黑盒化的对话历史堆叠。

这种显式设计带来了两个关键优势：第一，**state** 既是全局共享的，又可以通过私有字段实现局部隔离；第二，由于流转路径由图结构显式定义，复杂 Agent 的上下文 orchestration 变得可追踪、可复现、可调试。

7.3 Mechanism：LangGraph 如何支撑四大策略

下面我们将逐一对应四大策略，说明 LangGraph 提供的原生技术能力。

7.3.1 存储: store 与 state 的分层存储

LangGraph 为“存储上下文”提供了两层机制:

- **store (持久化存储):** 用于跨会话甚至跨 Agent 的长期记忆,天然支持 **Semantic**、**Episodic**、**Procedural** 三类记忆分类。常见实现包括 **InMemoryStore**、**PostgresStore** 等。
- **state (临时状态):** 绑定于单次会话或单个图运行周期,扮演类似 Scratchpad 的热存储角色。通过 **MemorySaver** 等 **checkpoint**, 可以在会话中断后恢复当前状态。

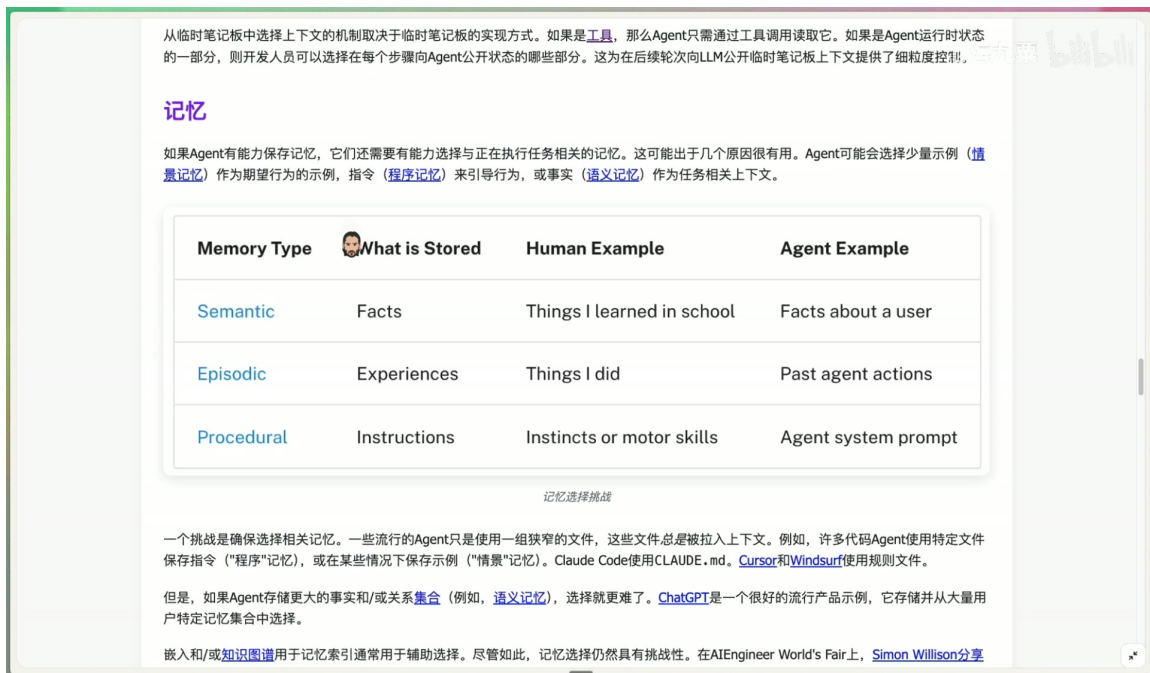


图 9: Memory 的三类分类: Semantic、Episodic、Procedural, LangGraph 的 store 为其提供持久化支撑⁷

```
1 from langgraph.checkpoint.memory import MemorySaver
2 from langgraph.store.memory import InMemoryStore
3
4 # store 用于持久化记忆 (跨会话、跨 Agent)
5 store = InMemoryStore()
6
7 # checkpoint 用于临时状态的保存与恢复
8 memory = MemorySaver()
```

Listing 8: LangGraph 中 store 与 checkpoint 的伪代码示意

7.3.2 选择: 全局 state 与私有 state 的精准投喂

在选择上下文方面, LangGraph 的节点机制具有独特优势:

- **全局 state:** 所有节点都可以访问共同的全局状态,这意味着一旦将外部知识或工具结果写入 state, 任何下游节点都可以直接读取, 实现跨节点的上下文共享。

⁷ 视频画面时间区间: 00:08:20–00:08:28。若为裁剪图, 时间区间仍对应原视频画面。

- **私有 state:** LangGraph 允许节点拥有不对外暴露的私有状态字段。开发者可以精确控制“哪个节点看到哪些内容”，从而避免无关信息挤占特定节点的上下文窗口。

正如视频中强调的，LangGraph 的数据流转“比较可控的”——它既是一个选择 (Select) 的过程保障，又是一个天然的数据隔离与分而治之的保障。

7.3.3 压缩: trim_messages、summarize 与自定义清理

LangGraph 及其生态为压缩上下文提供了丰富的内置工具与扩展接口：

- **工具函数:** 如 trim_messages 可直接按 token 阈值裁剪消息列表；
- **LangMem:** 专门用于对话历史的摘要与压缩，提供 summarize_history 等功能；
- **自定义逻辑:** 由于 state 的完全透明性，开发者可以在任意节点中插入自己的压缩、过滤或 TTL 清理逻辑。

```

1 from langchain.memory import trim_messages
2
3 # 概念示意：在节点中按 token 上限裁剪历史消息
4 def compress_node(state):
5     state["messages"] = trim_messages(
6         state["messages"],
7         max_tokens=4000,
8         strategy="last"
9     )
10    return state

```

Listing 9: 在 LangGraph 节点中自定义压缩逻辑的伪代码示意

7.3.4 隔离: subgraph、Tool Node 与 sandbox

当单窗口容量达到极限时，LangGraph 提供了多重隔离手段：

- **Subgraph (子图):** 每个子图拥有独立的局部 state，只接收与其任务相关的子集。这与分布式计算中将任务分发到多台机器各自的 RAM 中处理的理念完全一致。
- **Tool Node:** LangGraph 提供原生的工具调用节点，将工具输入输出隔离在特定字段中，避免工具描述和返回结果无序地膨胀全局上下文。
- **Sandbox:** 对于需要运行大量代码或处理大文件的子任务，可以通过沙盒节点在隔离环境中执行，仅将最终结果回写至上层 state。

7.4 LangSmith Trace: 调试上下文流转的可视化抓手

在复杂的 LangGraph 应用中，上下文可能经历了存储、选择、压缩、隔离的多次变换，单纯靠打印日志很难追踪其全貌。这时，LangSmith 的 trace 能力就显得至关重要。通过 LangSmith，开发者可以清晰地看到：

- 每一步模型的真实输入与输出；

- 各个时刻 `state` 的完整快照；
- 内存使用量的变化曲线，以及消息列表的长度变化；
- 工具调用的详细链路及其对上下文的贡献。

核心概念

LangSmith 的 `trace` 是调试复杂上下文流转的“X 光片”。当 Agent 输出异常时，首要排查的不是 `prompt` 写得是否优美，而是 `trace` 中显示的上下文是否包含了不该包含的内容，或遗漏了关键信息。

正如视频中所建议的，实际开发中“最终能看得比较清楚的就是一个 `trace` 的链路”。利用 LangSmith 观察真实问题，是将理论框架转化为可迭代工程实践的第一步。

7.5 Takeaway：把上下文策略写成可运行的状态图

LangGraph 的价值在于：它将 Context Engineering 的抽象策略翻译成了可直接编排的代码结构。当我们用“上下文工程”的透镜去审视 LangGraph 时，会发现它的每一个核心 API 都在回答同一个问题——如何在有限的上下文窗口中，为下一步操作填充恰到好处的好处内容？

- `store / state` 回答了“如何存”；
- 全局与私有 `state` 回答了“如何选择”；
- `trim_messages / summarize` / 节点内自定义逻辑回答了“如何压缩”；
- `subgraph / Tool Node / sandbox` 回答了“如何隔离”。

7.6 本章小结

本节系统梳理了 LangGraph 对上下文工程四大策略的原生支持：

- **存储**：通过 `store` 实现 Semantic / Episodic / Procedural 持久记忆，通过 `state` 与 `MemorySaver` 实现临时热存储。
- **选择**：全局 `state` 支撑跨节点共享，私有 `state` 实现精细化、节点级别的上下文投喂。
- **压缩**：内置 `trim_messages`、LangMem 摘要工具，以及 `state` 透明性带来的自定义压缩能力。
- **隔离**：`subgraph` 提供状态隔离，`Tool Node` 与 `sandbox` 提供执行环境隔离。
- **可观测性**：LangSmith `trace` 是追踪上下文流转、调试存储/选择/压缩/隔离逻辑的关键工具。

8 结论：Context Engineering 的系统思维与下一步实践

8.1 Motivation：Agent 效能的真正瓶颈在哪里

回顾开篇提到的 Gavin 与 Anthropic 之争，以及 `LLM = CPU`，`Context Window = RAM` 的核心隐喻，我们会发现一个反复出现的主题：无论我们争论的是单 Agent 还是多 Agent、大模型

还是小模型，真正决定输出质量的往往不是模型本身的参数量，而是进入上下文窗口的那“几 K 到几百 K token”的质量。

核心概念

Agent 开发的竞争焦点正在从“模型选择”转向“上下文管理”。一个参数量较小的模型，配上精准的上下文工程，往往能在实际任务中击败一个被垃圾信息淹没的大模型。

8.2 Idea：四大策略的协同闭环

上下文工程的四大策略不是孤立拼凑的技巧，而是一个彼此咬合、协同增效的完整系统：

- **存储（Write） + 选择（Select） = 精准投喂**：先把有价值的信息 structured 地存好，再在恰当的时机捞出最相关的内容喂给模型。
- **压缩（Compress） = 清理空间**：即使精心挑选的内容，也可能因体积过大而需要进一步摘要或裁剪，为新的推理留出余量。
- **隔离（Isolate） = 横向扩展**：当单窗口的硬边界终究无法容纳任务复杂度时，通过子图、子 Agent 或 Sandbox 将上下文“分而治之”是唯一根本性的扩展之道。

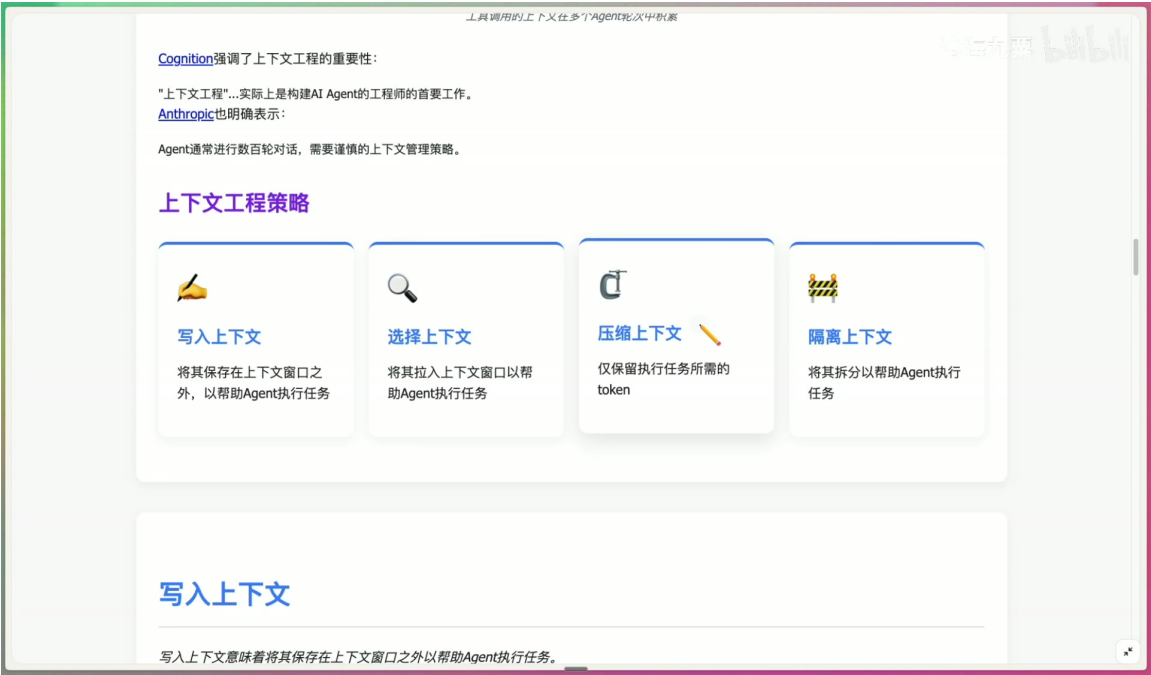


图 10: 上下文工程的四大策略闭环：Write、Select、Compress、Isolate 共同构成 Agent 效能优化的完整系统⁸

8.3 Mechanism：从局部优化到系统思维

系统思维的转折点在于，我们不再追问“这个任务该用哪个更强的模型”，而是追问“这个任务需要怎样的上下文 orchestration”。通过合理组合四大策略，我们可以在不更换模型、甚至不增加

⁸视频画面时间区间：00:04:20–00:04:28。若为裁剪图，时间区间仍对应原视频画面。

token 预算的前提下，显著提升 Agent 的推理质量与稳定性——整体上下文的丰富度没有丢失，但每一个推理步骤、每一个子 Agent 只拿到最精炼、最相关的局部上下文，从而实现 $1+1>2$ 的效果。

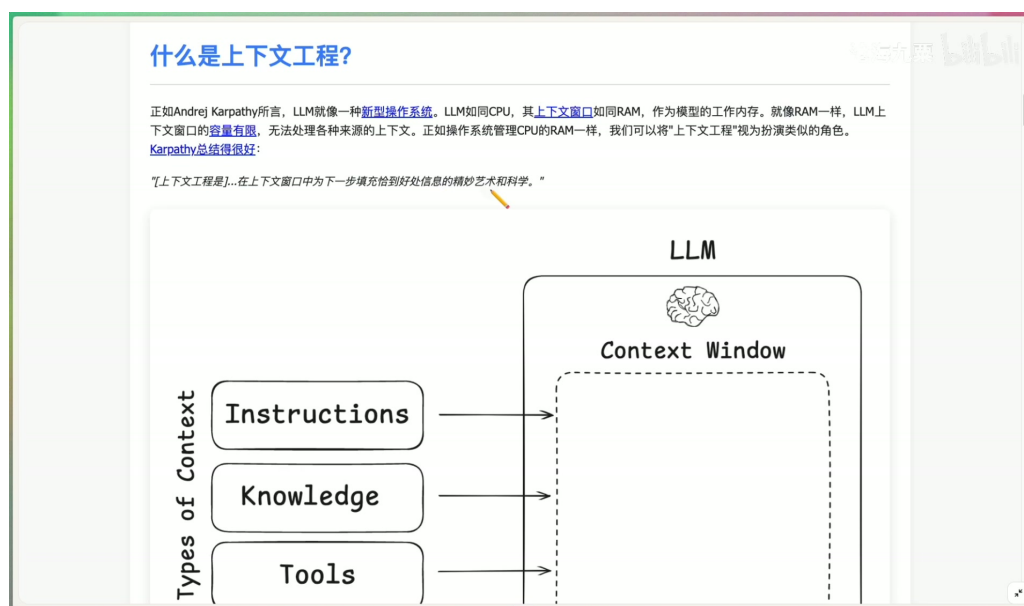


图 11: 首尾呼应：LLM 作为 CPU，Context Window 作为 RAM，Context Engineering 就是操作系统对内存的精细化管理⁹

正如 CPU 的运算能力再强，没有高效的内存管理也无法发挥全部性能；LLM 的推理能力再强，如果缺乏系统的上下文工程，也同样会被无关信息淹没，陷入 *garbage in, garbage out* 的困境。

8.4 Takeaway：下一步实践建议

视频最后强调了一个重要的实践原则：**理论总是比较抽象的，最终还要在大家的实践中发现问题**。要将 Context Engineering 从纸面框架转化为工程能力，我们建议从以下三步开始：

1. **用 LangSmith 观察真实问题**：不要凭直觉猜测上下文瓶颈，先打开 trace，观察模型输入、state 快照和内存使用的真实数据。
2. **从单一策略入手，逐步组合**：可以先解决最突出的问题（例如工具返回值过长先压缩，或历史消息过多先 trim），再逐步将存储、选择、隔离策略纳入统一设计中。
3. **培养系统思维**：每次遇到 Agent 输出不佳时，第一反应不再是“换模型”或“调 prompt”，而是检查“当前上下文窗口里到底装了什么”。

未来的 Agent 开发竞争，必将从“谁的模型更大”转向“谁的上下文管理更优雅”。掌握 Context Engineering 的系统思维，善用 LangGraph 的原生支持，并通过 LangSmith 持续观察与迭代，才能真正建造出高效、可扩展、值得信赖的智能体系统。正如 Andrew Karpathy 所言：上下文工程，正是在上下文窗口中为下一步操作填充恰到好处的内容的精妙艺术和科学——这门艺术，值得每一位 Agent 开发者深入学习与实践。

⁹视频画面时间区间：00:02:37–00:02:45。若为裁剪图，时间区间仍对应原视频画面。